

<HLS Programming with FPGAs>

The HLS Book

Chapter 7

Dense Matrix Multiplication 2

Choi, Young-kyu

Recap

- Pipelining loop J:

```
void matrixmul(int A[N][M], int B[M][P], int AB[N][P]) {  
    #pragma HLS ARRAY_PARTITION variable=A complete dim=2  
    #pragma HLS ARRAY_PARTITION variable=B complete dim=1  
    /* for each row and column of AB */  
    row: for(int i = 0; i < N; ++i) {  
        col: for(int j = 0; j < P; ++j) { //pipelined  
            #pragma HLS PIPELINE II=1  
            /* compute (AB)i,j */  
            int ABij = 0;  
            product: for(int k = 0; k < M; ++k) { //unrolled  
                ABij += A[i][k] * B[k][j];  
            }  
            AB[i][j] = ABij;  
        }  
    }  
}
```

//ARRAY_RESHAPE is possible

- M MAC operations per cycle
- Needs to partition A and B by M

Recap

- Data reuse in matrix multiplication:

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & \cdots & B_{1p} \\ B_{21} & B_{22} & \cdots & B_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ B_{m1} & B_{m2} & \cdots & B_{mp} \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} (AB)_{11} & (AB)_{12} & \cdots & (AB)_{1p} \\ (AB)_{21} & (AB)_{22} & \cdots & (AB)_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ (AB)_{n1} & (AB)_{n2} & \cdots & (AB)_{np} \end{bmatrix}$$

→ A_{11} can be reused up to **p** times if stored in internal memory

- Challenge: How can we exploit this data reuse even if the entire matrices **cannot fit** inside the internal memory?

Learning Objectives

- Explain the advantage of blocked matrix multiplication
- Optimize blocked matrix multiplication implementation

7.3 Block Matrix Multiplication

- Idea: Partition a matrix into smaller blocks and process only a block of a matrix at a time
- Example:

A_{11}	A_{12}	A_{13}	A_{14}
A_{21}	A_{22}	A_{23}	A_{24}
A_{31}	A_{32}	A_{33}	A_{34}
A_{41}	A_{42}	A_{43}	A_{44}

×

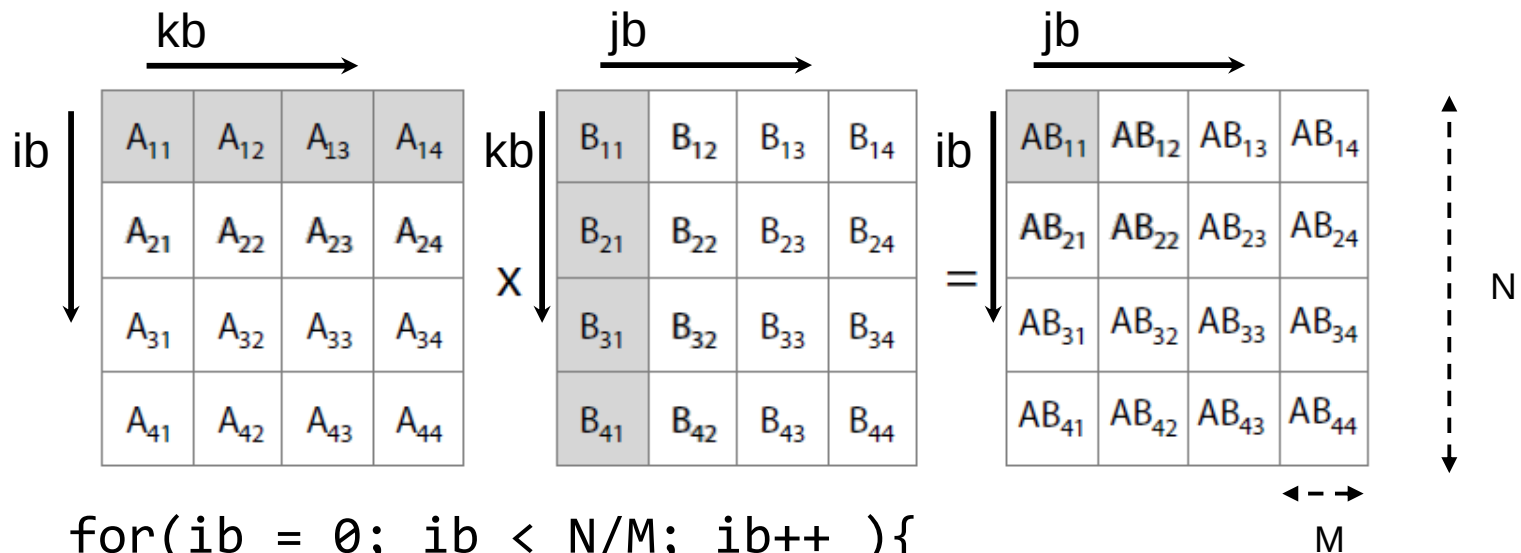
B_{11}	B_{12}	B_{13}	B_{14}
B_{21}	B_{22}	B_{23}	B_{24}
B_{31}	B_{32}	B_{33}	B_{34}
B_{41}	B_{42}	B_{43}	B_{44}

=

AB_{11}	AB_{12}	AB_{13}	AB_{14}
AB_{21}	AB_{22}	AB_{23}	AB_{24}
AB_{31}	AB_{32}	AB_{33}	AB_{34}
AB_{41}	AB_{42}	AB_{43}	AB_{44}

- $AB_{11} = A_{11}B_{11} + A_{12}B_{21} + A_{13}B_{31} + A_{14}B_{41}$
- Only needs to store 3 submatrices (e.g., A_{12} , B_{21} , AB_{11}) in the internal memory at a time
- Can still exploit data reuse

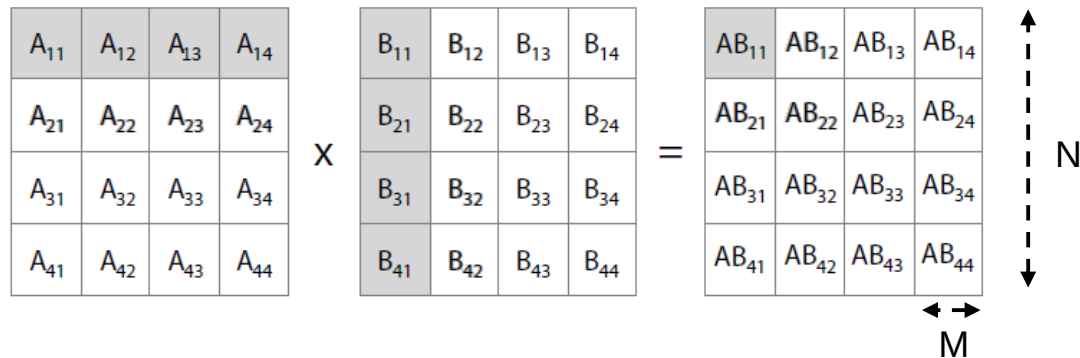
- Suppose we are multiplying matrices of size $N \times N$, and partitioning into blocks $M \times M$
 - Loop ordering among blocks



```
for(ib = 0; ib < N/M; ib++) {
    for(jb = 0; jb < N/M; jb++) {
        for(kb = 0; kb < N/M; kb++) {
            // do computation within a block
        }
    }
}
```

– DRAM data transfer size

- $AB : M * M * (N/M) * (N/M) = N^2$
- $A : M * M * (N/M) * (N/M) * (N/M) = N^3/M$
- $B : N^3/M$



- Total transfer size $\approx 2 * N^3/M$ ($N \gg M$)
- cf) Total transfer size when not blocked $\approx 2 * N^3$

→ DRAM transfer reduced by M times using internal memory (size depends on loop ordering within a block)

- Loop ordering within a block

- Ex) loop order $i \rightarrow j \rightarrow k$:

... (see previous slide for block loops) ...

```
for(int i = 0; i < M; i++) {  
    for(int j = 0; j < M; j++) {  
        DTYPE AB_temp = 0;  
        for(int k = 0; k < M; k++) {  
            AB_temp += A_block[i][k] * B_block[k][j];  
        }  
        AB_block[i][j] = AB_temp;  
    }  
}
```

- Quiz 1) Can we interchange I and J loops?
 - Yes
 - Quiz 2) Can we interchange J and K loops?
 - No, but it is possible with some code modification

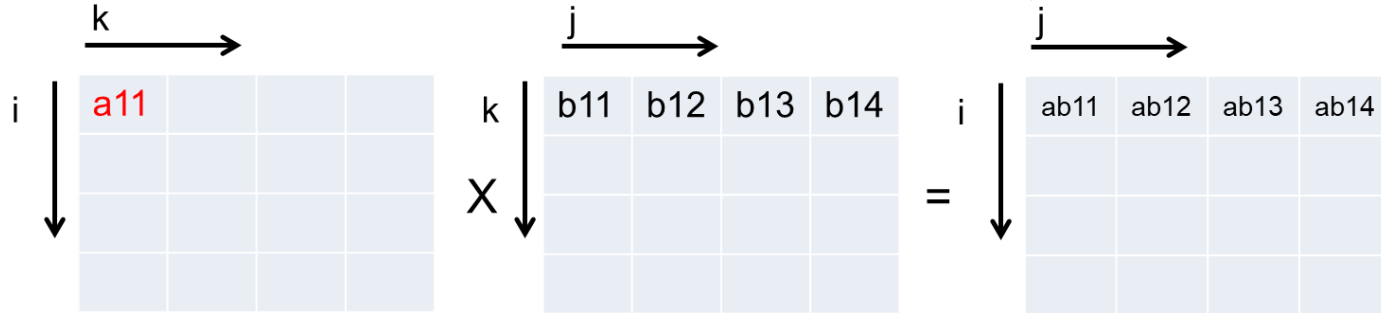
- Loop ordering within a block (...continued)
 - Ex) loop order $i \rightarrow k \rightarrow j$:

```
... (see previous slide for block loops) ...  
  
for(int i = 0; i < M; i++) {  
    for(int j = 0; j < M; j++) {  
        AB_block[i][j] = 0;  
    }  
    for(int k = 0; k < M; k++) {  
        for(int j = 0; j < M; j++) {  
            AB_block[i][j] += A_block[i][k] * B_block[k][j];  
        }  
    }  
}
```

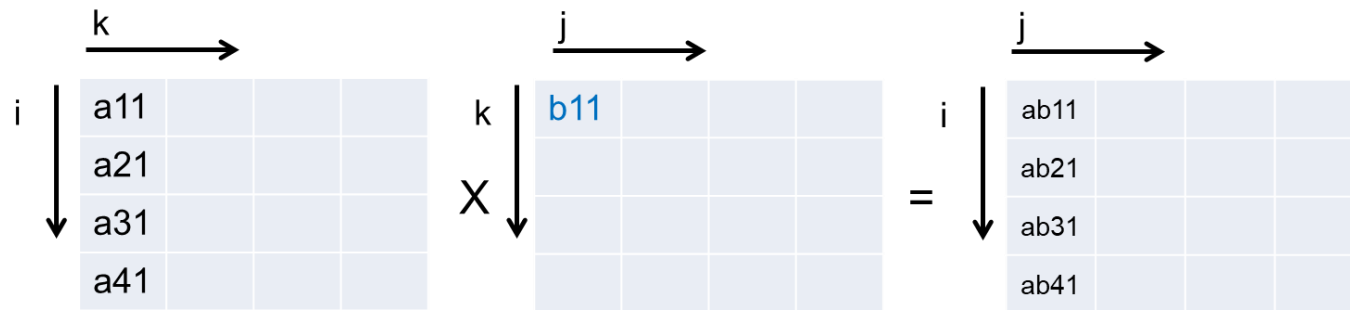
- We can change the loop ordering with proper initialization.
- Q) Which loop ordering should we use?

– Loop ordering decision factor 1: **Data reuse and internal memory usage**

- Element **a11** can be reused across loop j



- Element b11 can be reused across loop i



- If loop order is $k \rightarrow i \rightarrow j$
 - a11 is reused in the next iteration (reuse distance = 1), and b11 is reused after M iterations (reuse distance = M)
- If loop order is $k \rightarrow j \rightarrow i$
 - a11 is reused after M iterations (reuse distance = M), and b11 is reused in the next iteration (reuse distance = 1)

– Loop ordering and internal memory usage

- If loop order is $k \rightarrow i \rightarrow j$

- Internal memory size for A : 1 ($\because$ reuse distance = 1)
- Internal memory size for B : M ($\because$ reuse distance = M)

- If loop order is $k \rightarrow j \rightarrow i$

- Internal memory size for A : M ($\because$ reuse distance = M)
- Internal memory size for B : 1 ($\because$ reuse distance = 1)

Similar mem usage

- If loop order is $i \rightarrow j \rightarrow k$

- Internal memory size for A : M ($\because$ reuse distance = M)
- Internal memory size for B : M^2 ($\because$ reuse distance = M^2)

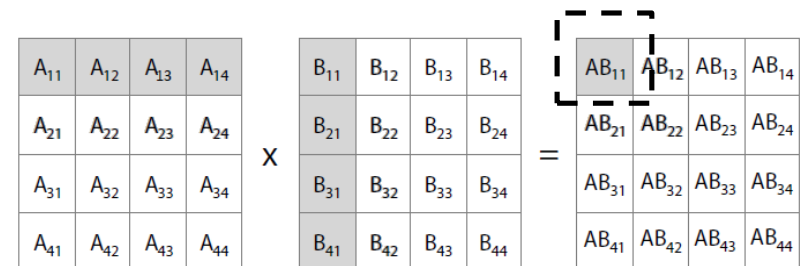
Large mem usage ☹

– Internal memory usage (assuming loop order $k \rightarrow i \rightarrow j$)

- A : 1

- B : M (line)

- AB : M^2 (block)

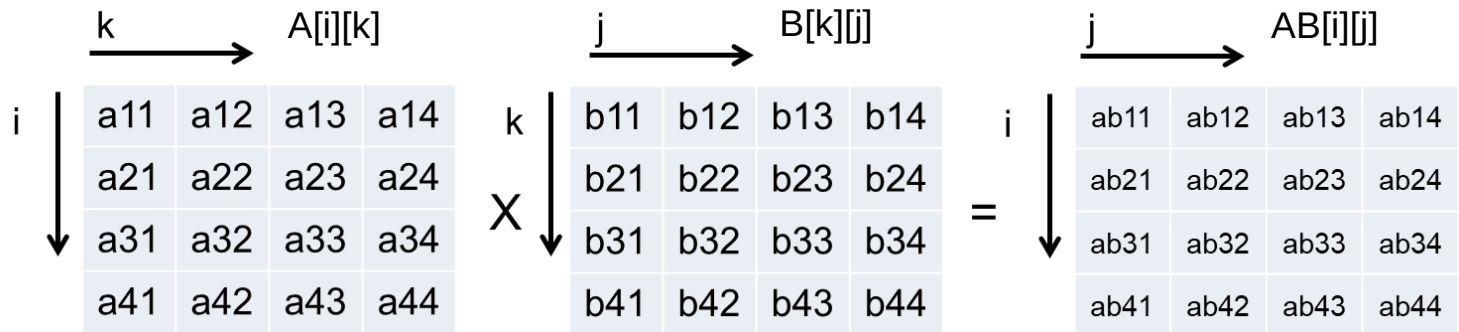


- Comparison between non-blocked vs blocked MM

	Internal mem	DRAM transfer
MM, non-blocked	0	$\sim 2*N^3$
MM, blocked	$\sim M^2$	$\sim 2*N^3/M$

- Larger internal memory usage for reduced DRAM transfer

- Loop ordering decision factor 2: **DRAM access pattern**
 - Data is stored in row major in most systems
 - DRAM effective BW is improved if the innermost loop accesses the same row



- If loop order is $\mathbf{k \rightarrow i \rightarrow j}$
 - A : Column traversal ☹, B and C: Row traversal ☺
- If loop order is $\mathbf{k \rightarrow j \rightarrow i}$
 - A and B and C : Column traversal ☹
- If loop order is $\mathbf{i \rightarrow j \rightarrow k}$
 - A and C : Row traversal ☺, B : Column traversal ☹

Baseline Blocked MM Implementation

- Create a Vitis project for MM
 - Please refer to the previous lecture "Vitis and Vitis HLS Flows"
 - Copy project from Hello World
 - For host file
 - Create memory objects
 - Set kernel arguments
 - Make PCIE transfer
 - Launch kernel
 - For Makefile
 - Change the directory name & kernel name to match dense MM

- Baseline kernel code:

```

typedef short DTYPE;
const int M = 256;
//M is block size, and N is matrix size

extern "C" {
    void mm(DTYPE* A, DTYPE* B, DTYPE* AB, int N ) {
        #pragma HLS INTERFACE mode=m_axi bundle=m0 port=A
        #pragma HLS INTERFACE mode=m_axi bundle=m1 port=B
        #pragma HLS INTERFACE mode=m_axi bundle=m1 port=AB

        DTYPE AB_block[M][M];
        DTYPE B_line[M];

        ..... (see next slide ) .....
    }
}

```

pointer argument

AXI interface

Internal memory
(for $K \rightarrow I \rightarrow J$ loop order)

- Baseline kernel code (...continued):
 - Assumes $k \rightarrow i \rightarrow j$ loop order inside a block

..... (see previous slide)

```

for(int ib = 0; ib < N; ib++) {
  for(int jb = 0; jb < N; jb++) {
    // Initialize AB_block[i][j] to 0 here
    for(int kb = 0; kb < N; kb++) {

      for(int k = 0; k < M; k++) {
        for(int j = 0; j < M; j++) {
          B_line[j] = B[(kb*M+k)*N + jb*M + j];
        }
        for(int i = 0; i < M; i++) {
          DTYPE Atemp = A[(ib*M+i)*N + kb*M + k];
          for(int j = 0; j < M; j++) {
            AB_block[i][j] += Atemp * B_line[j];
          }
        }
      }
    }
  }

  // Copy AB_block[i][j] to DRAM port AB here
}

```

1D array addressing

- Design parameters
 - Data type = short
 - N (matrix size) = 512
 - M (block size) = 256
 - Please run on-board test using Vitis
 - Baseline performance
 - Execution time: 5.5s
 - Performance = $2 * 512 * 512 * 512 / 5.5 = 0.049$ GOPS
- : Very bad performance for a design without any pragmas

- See Vitis synthesis log (_x.hw_emu.xilinx_aws-vu9p-f1_shell-v04261818_201920_2/mm/mm/vitis_hls.log):

```
INFO: [XFORM 203-541] Flattening a loop nest 'i_loop' (/home/centos/src/project_data/aws-fpga/Vitis/examples/xilinx_2021.1/mm_onboard/src/mm.cpp:33:22) in function 'mm'.
INFO: [XFORM 203-510] Pipelining loop 'j_loop' (/home/centos/src/project_data/aws-fpga/Vitis/examples/xilinx_2021.1/mm_onboard/src/mm.cpp:35) in function 'mm' automatically.
INFO: [HLS 200-10] -----
INFO: [HLS 200-42] -- Implementing module 'mm_Pipeline_i_loop_j_loop'
INFO: [HLS 200-10] -----
INFO: [SCHED 204-11] Starting scheduling ...
INFO: [HLS 200-486] Changing DSP48 latency (root=mul_ln36) to 3 in order to utilize available DSP registers.
INFO: [SCHED 204-61] Pipelining loop 'i_loop_j_loop'.
INFO: [HLS 200-1470] Pipelining result : Target II = NA, Final II = 1, Depth = 80, loop 'i_loop_j_loop'
INFO: [SCHED 204-11] Finished scheduling.
INFO: [HLS 200-111] Finished Scheduling: CPU user time: 0.2 seconds. CPU system time: 0 seconds. Elapsed time: 0.2 seconds; current allocated memory: 167.355 MB.
```

- Vitis has automatically pipelined J loop, and combined (flattened) I and J loops → not enough parallelism

```
i_loop: for(int i = 0; i < M; i++) {
    DTYPE Atemp = A[(ib*M+i)*N + kb*M + k];
    j_loop: for(int j = 0; j < M; j++) {
        AB_block[i][j] += Atemp * B_line[j];
    }
}
```

Opt1: Pipelined & Unrolled Blocked MM

- Optimization strategy:
 - Unroll j loop, pipeline i loop

```
.....  
    for(int i = 0; i < M; i++) {  
#pragma HLS pipeline II=1  
        DTYPE Atemp = A[(ib*M+i)*N + kb*M + k];  
        for(int j = 0; j < M; j++) {  
#pragma HLS unroll  
            AB_block[i][j] += Atemp * B_line[j];  
        } }  
.....
```

- Ideally, performs M MAC operations per cycle

- Internal memory needs to have enough ports to support loop unrolling
- Partitioning of internal memory:

```

.....
DTYPE AB_block[M][M];
DTYPE B_line[M];
#pragma HLS ARRAY_PARTITION dim=2 type=complete variable=AB_block
#pragma HLS ARRAY_PARTITION dim=1 type=complete variable=B_line
.....
    for(int i = 0; i < M; i++) {
#pragma HLS pipeline II=1
        DTYPE Atemp = .....;
        for(int j = 0; j < M; j++) {
#pragma HLS unroll
            AB_block[i][j] += Atemp * B_line[j];
        }
    }
.....

```

- Quiz:
 - Why must M (block size) be a constant, and why can N (matrix size) be a scalar variable?

```
const int M = 256;
.....
void mm(DTYPE A[...], B[...], AB[...], int N){
.....
DTYPE AB_block [M][M];
DTYPE B_line [M];
.....
for(int ib = 0; ib < N / M; ib++) {
    for(int jb = 0; jb < N / M; jb++) {
.....
        for(int i = 0; i < M; i++) {
#pragma HLS pipeline II=1
            for(int j = 0; j < M; j++) {
#pragma HLS unroll
                AB_block[i][j] += Atemp * B_line[j];
.....
            }
        }
    }
}
```

- Performance after pipelining and unrolling
 - Execution time: 5.7ms
 - Performance: 47 GOPS
 - Speedup: $47/0.049 = 960X$
- Comparison with ideal performance
 - Ideal performance = $2 * 256 * 0.250\text{GHz} = 128 \text{ GOPS}$
 - 47 GOPS -> only 36% of the ideal performance

- Reason for the low performance : Low effective DRAM BW

Can only copy 16b per cycle

```
void mm(DTYPE* A, DTYPE* B, DTYPE* AB, int N ){  
#pragma HLS INTERFACE mode=m_axi bundle=m0 port=A  
#pragma HLS INTERFACE mode=m_axi bundle=m1 port=B  
#pragma HLS INTERFACE mode=m_axi bundle=m1 port=AB  
  
}
```

- Cannot provide enough data to utilize all MAC units
- Needs DRAM optimizations

Summary

- Blocked matrix multiplication helps reduce the DRAM access by reusing the data stored in the internal memory
- We should choose a loop ordering that consumes a small internal memory and improves the DRAM access bandwidth
- We can apply pipelining, unrolling, and array partitioning pragmas for high performance